

Ввод в создание чат-ботов

Главная цель чат-бота – максимально эффективно выполнить свое предназначение при общении с пользователем.

Основные требования к чат-боту:

- 1) Соответствие ответов поставленной задаче или теме чата
- 2) Своевременность ответов
- 3) Интуитивно понятное взаимодействие с ботом
- 4) Способность бота к многозадачности

Соответствие ответов поставленной задаче или теме чата – это важная черта, влияющая на адекватность бота. К примеру, если бот для продажи репчатого лука начнет рассказывать, как правильно натягивать тетиву, то покупатели перестанут обращаться к данному боту.

Своевременность ответов больше относится к стабильной работе программы, и правильности настройки бота. Это означает, что бот должен работать стабильно и быть способным отвечать на большинство поставленных вопросов.

Интуитивно понятное взаимодействие с ботом. Взаимодействие с ботом – как взаимодействие с графическим интерфейсом должно быть понятно пользователю без длительных пояснений. В этом поможет упрощение команд, создание списка команд, создание кнопок в чате.

Способность бота к многозадачности – способность бота обрабатывать запросы от множества пользователей в один промежуток времени. В этом может помочь асинхронность.

Особенности работы с Aiogram:

- 1) Вся библиотека Aiogram асинхронная, что позволяет ей выполнять множество команд параллельно.
- 2) Большинство функций бота являются реакцией на действия в чате, для распознавания действий требуется применение хэндлеров, роутеров и диспетчера.
- 3) Бот создается самостоятельно

Диспетчер – часть программы постоянно проверяющая наличие новых сообщений и передающая их в соответствующий хэндлер.

Роутер – часть программы аналогичная диспетчеру, однако отвечающая за группу хэндлеров.

Хэндлер(Handler) – Обработчик полученных от диспетчера или роутера данных. Анализирует полученный текст, проверяет его по внутренним условиям и выполняет функцию, соответствующую одному или нескольким из внутренних условий.

Последовательность работы программы, следующая:

- 1) Пользователь пишет в чат или применяет команду
- 2) Диспетчер или роутер воспринимают текст или команду, анализируют и отправляют соответствующему хэндлеру, также передают данные внутри команды.
- 3) Хэндлер принимает данные и выполняет заложенные в нем функции, например пишет ответное сообщение.
- 4) Бот выдает реакцию на полученную команду в чат. (Этот пункт может отсутствовать на усмотрение разработчика)

Обработка сообщений

Сообщения Telegram бывают трех видов:

- 1) Текстовые
- 2) Медиафайлы
- 3) Служебные

Обработка сообщений проходит одним и тем же образом для всех видов сообщений:

- 1) Прием сообщения
- 2) Предобработка сообщений при помощи диспетчера
- 3) Распределение сообщений по хэндлерам при помощи диспетчера
- 4) Вызов функции хэндлера
- 5) Обработка сообщения в функции
- 6) Отправка реакции в чат

Обработка текстовых сообщений

Текстовые сообщения могут быть восприняты как команды или текст.

Команды воспринимаются при помощи функции `Command("команда")`, при этом у команд есть префикс, стандартно префикс `"/"`.

Изменить префикс можно дополнительным атрибутом `prefix`

`Command("команда",prefix="префикс")`

Пример:

```
@dp.message(Command("start",prefix="%"))
```

Помимо команд чат-бот может принимать и обрабатывать текстовые сообщения:

Для этого в хендлере прописывается Атрибут F.text

Пример:

```
from aiogram import Bot, Dispatcher, types, F
```

```
@dp.message(F.text)
```

```
async def cmd_start(message: types.Message):
```

```
    await message.answer("is text")
```

Если внутри хендлера F.text сравнить с текстом, то чат-бот будет откликаться на сообщение с соответствующим текстом. Остальные сообщения игнорируются

```
@dp.message(F.text=="глад")
```

Также можно проверять текст по началу startswith(""), по окончанию endswith(""), по наличию контента contains("")

Пример:

```
@dp.message(F.text.startswith("Это"))
```

```
@dp.message(F.text.endswith("То"))
```

```
@dp.message(F.text.contains("содержит"))
```

Можно объединять условия в одно:

Логика объединения как с логическим И(&):

```
@dp.message(F.text.startswith("Это") & F.text.contains("содержит"))
```

Логика объединения как с логическим ИЛИ(|):

```
@dp.message(F.text.startswith("Это") | F.text.contains("содержит"))
```

Логiku можно комбинировать любым образом, как обычное условие if.

Обработка медиафайлов

Медиафайлами являются фотографии и видеозаписи.

Отличие фотографий:

Определение фотографий происходит за счет атрибута `F.photo` в диспечере

Пример

```
@dp.message(F.photo)
```

Определение анимации происходит за счет атрибута `F.animation` в диспечере

Пример

```
@dp.message(F.animation)
```

Определение музыки происходит за счет атрибута `F.audio` в диспечере

Пример

```
@dp.message(F.audio)
```

Определение голосовые сообщения происходит за счет атрибута `F.voice` в диспечере

Пример

```
@dp.message(F.voice)
```

Определение документов происходит за счет атрибута `F.document` в диспечере

Пример

```
@dp.message(F.document)
```

Как вы заметили применяется фильтр `F`, полученные фотографии и видео можно обработать, преобразовать или конвертировать. Однако данный функционал выполняется поверх библиотеки `aiogram`.

Обработка специальных сообщений

Специальные сообщения – это сообщения о выполнении действий внутри системы Telegram, например добавление или удаление пользователя, изменение сообщения. Также возможно идентифицировать пользователя по ID.

Реакция на добавление участника в беседу:

F.new_chat_members

@dp.message(F.new_chat_members)

Реакция на уход пользователя из беседы left_chat_members:

@dp.message(F.left_chat_members)

Изменение и удаление фотографий беседы new_chat_photo, delete_chat_photo

@dp.message(F.new_chat_photo)

@dp.message(F.delete_chat_photo)

Реакция на изменение названия в беседы, new_chat_title:

@dp.message(F.new_chat_title)

Способы отправить сообщение

Способы сообщений отправить сообщения есть трех видов:

1) Обычные сообщения или ответ на сообщение

3) Сообщение с медиафайлами, стикерами и эмоджи. Могут быть применены обычные сообщения или ответы на вопросы

Обычное отправление сообщение вам знакомо:

Команда

message.answer("Сообщение отправлено")

Ответ на предыдущее сообщения – пересылка на предыдущего сообщения с дополнением текста в качестве сообщения.

message.reply("Ответ на сообщение отправлен")

Отправление

answer_dice() – отправляет сообщение с анимированным объектом отображение которого зависит от генератора случайных чисел, варианты символов '🎲', '🎯', '🏀', '🎱', '🎮', or '🎮'

reply_dice() – отправляет ответ на сообщение с анимированным объектом отображение которого зависит от генератора случайных чисел, варианты символов '🎲', '🎯', '🏀', '🎱', '🎮', or '🎮'

`answer_sticker('emogy')` – отправляет эмодзи записанный в "

`reply_sticker('emogy')` – отвечает на предыдущее сообщение при помощи эмодзи

`anwer_photo()` – отправляет фото по ссылке расположенное в файловой системе или по внешней ссылке

`reply_photo()` – отправляет фото расположенное в файловой системе или по внешней ссылке

`anwer_video()` – отправляет видео расположенное в файловой системе или по внешней ссылке

`reply_video()` – отправляет видео расположенное в файловой системе или по внешней ссылке

`answer_audio()` – отправляет аудиофайл расположенный в файловой системе или по внешней ссылке

`reply_audio()` – отправляет аудиофайл расположенный в файловой системе или по внешней ссылке

`answer_voice()` – отправляет аудиофайл в виде голосового сообщения. Файл расположен в файловой системе или по внешней ссылке

`reply_voice()` – отправляет ответ на сообщение, аудиофайл в виде голосового сообщения. Файл расположен в файловой системе или по внешней ссылке

`answer_animation()` - отправляет анимацию. Файл расположен в файловой системе или по внешней ссылке

`reply_animation()` - отправляет анимацию в виде ответа. Файл расположен в файловой системе или по внешней ссылке

Для использования файловой системы необходимо использовать функцию

```
types.FSInputFile('ссылка на файл');
```

Для использования файлов по URL следует применить функцию:

```
types.URLInputFile('ссылка на файл через https')
```

Подробнее с форматами можно ознакомиться здесь:

<https://docs.aiogram.dev/en/dev-3.x/modules/aiogram/types/message.html#>

Удаление и изменение сообщений (5 минут)

Для изменения и удаления сообщений следует сохранить объект отправленного сообщения или любого другого сообщения.

Сохранение объекта сообщения, которым ответили происходит следующим образом:

```
msg = await message.answer("First message")
```

Удаление и изменение сообщения применяется уже к сохраненному объекту, при том изменение сообщения можно применить только к отправленным ботом сообщений.

Удаление сообщения, delete:

```
await msg.delete()
```

Изменение сообщения, edit_text:

```
await msg.edit_text("Second message")
```